

SGA Rest Web Service



Guía: Creación del REST Web Service de la aplicación SGA

Introducción

En esta lección vamos a aprender cómo exponer un Web Service **RESTful** en nuestro proyecto `sga-jee-web`.

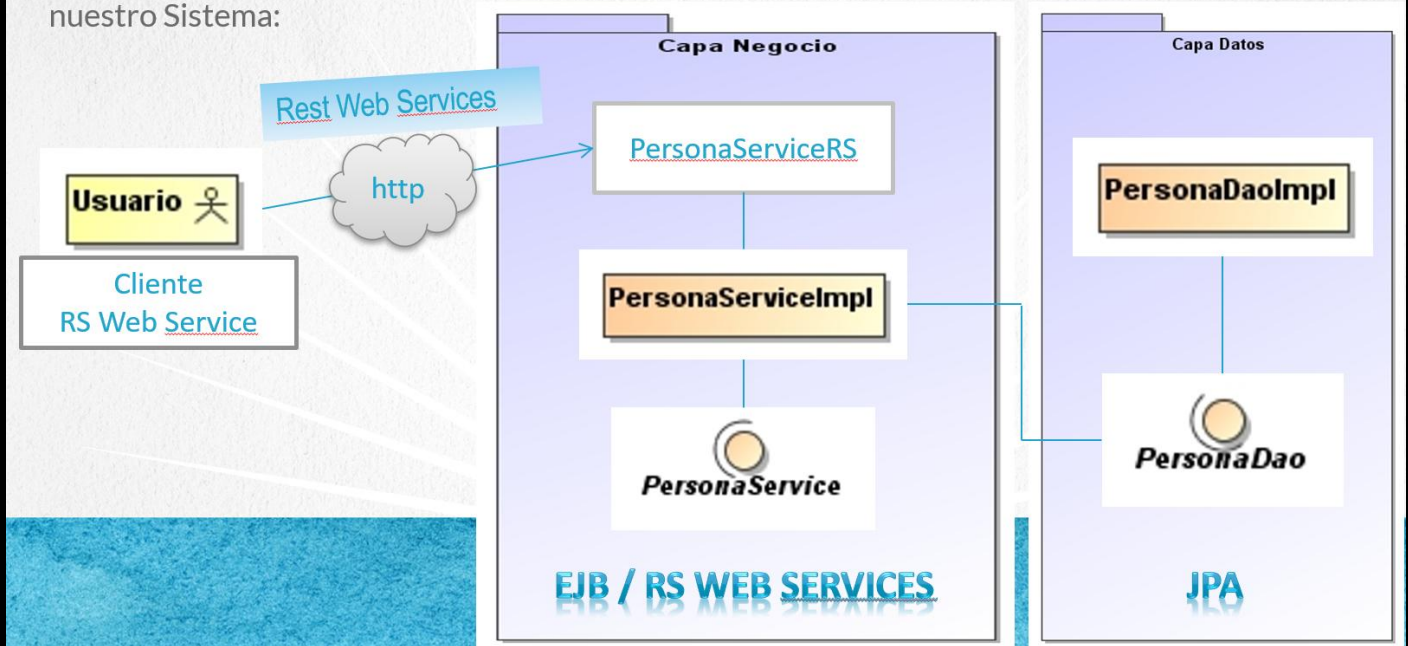
A diferencia de los Web Services tipo SOAP, este se basa en el protocolo **HTTP** y utiliza anotaciones como `@GET`, `@POST`, `@PUT` y `@DELETE` para manipular recursos.

Además, vamos a configurar correctamente la clase entidad `Persona` para que pueda enviarse como **XML** o **JSON** .

Esta es la arquitectura que vamos a manejar en este ejemplo:

Arquitectura sga con web services

Este es el Diagrama de Clases del Ejercicio, donde se pueden observar la Arquitectura de nuestro Sistema:



✓ Paso 1: Abrir el proyecto

📁 Si aún no está abierto, abre el proyecto `sga-jee-web` para empezar a trabajar esta nueva lección.

✓ Paso 2: Agregar la dependencia de Jersey en el archivo `pom.xml`

📄 Abre el archivo `pom.xml` y al final de la sección `<dependencies>`, agrega lo siguiente:

```

<dependency>
  <groupId>org.glassfish.jersey.core</groupId>
  <artifactId>jersey-client</artifactId>
  <version>3.1.10</version>
</dependency>

```

📦 Esta dependencia nos permitirá trabajar con el API de **JAX-RS** a través del servidor GlassFish.

Luego da clic derecho en el proyecto y selecciona 🖱️ "Clean and Build" para descargar la librería.

 Archivo pom.xml completo:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>sga</groupId>
    <artifactId>sga-jee-web</artifactId>
    <version>1.0</version>
    <packaging>war</packaging>
    <name>sga-jee-web</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <jakartaee>11.0.0-M1</jakartaee>
    </properties>

    <dependencies>
        <dependency>
            <groupId>jakarta.platform</groupId>
            <artifactId>jakarta.jakartaee-api</artifactId>
            <version>${jakartaee}</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>jakarta.persistence</groupId>
            <artifactId>jakarta.persistence-api</artifactId>
            <version>3.2.0</version>
        </dependency>
        <dependency>
            <groupId>org.eclipse.persistence</groupId>
            <artifactId>eclipselink</artifactId>
            <version>4.0.5</version>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>9.2.0</version>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
            <version>2.0.16</version>
            <type>jar</type>
        </dependency>
    </dependencies>
</project>
```

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-simple</artifactId>
  <version>2.0.16</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.primefaces</groupId>
  <artifactId>primefaces</artifactId>
  <version>15.0.0</version>
  <classifier>jakarta</classifier>
</dependency>
<dependency>
  <groupId>jakarta.jws</groupId>
  <artifactId>jakarta.jws-api</artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.core</groupId>
  <artifactId>jersey-client</artifactId>
  <version>3.1.10</version>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.12.1</version>
      <configuration>
        <source>21</source>
        <target>21</target>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-war-plugin</artifactId>
      <version>3.4.0</version>
    </plugin>
  </plugins>
</build>
</project>
```



Paso 3: Crear la clase `PersonaServiceRS.java`

📁 Ubicación: `src/main/java/sga/servicio/PersonaServiceRS.java`

Esta clase será la que **expone los métodos como Web Services REST**:



Paso a paso del contenido de la clase:

1. Agregar anotación `@Path`

```
@Path("/personas")
public class PersonaServiceRS {
```



Esto define el **endpoint base** del servicio (más adelante configuraremos el archivo `web.xml` donde se definirá la ruta completa, incluyendo `sga-jee-web/webservice`):

`http://localhost:8080/sga-jee-web/webservice/personas`

2. Inyectar el servicio de personas

```
@Inject
private PersonaService personaService;
```



Esto inyecta la lógica de negocio usando **CDI** (Context and Dependency Injection).

3. Método `listarPersonas`

```
@GET
@Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
public List<Persona> listarPersonas() {
    return personaService.listarPersonas();
}
```



Este método responde a una petición **GET** y devuelve una lista de personas en **XML o JSON** según lo que el cliente solicite.

4. Método `encontrarPersonaPorId`

```
@GET
@Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
```

```
@Path("{id}")
public Persona encontrarPersonaPorId(@PathParam("id") int id) {
    return personaService.encontrarPersonaPorId(new Persona(id));
}
```



Con esta URL puedes buscar una persona por su ID:

<http://localhost:8080/SGA-JEE-WEB/webservice/personas/1>

5. Método agregarPersona

```
@POST
@Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
@Consumes({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
public Response agregarPersona(Persona persona) {
    personaService.registrarPersona(persona);
    return Response.ok().entity(persona).build();
}
```



Este método permite enviar una persona vía JSON o XML para agregarla a la base de datos.

6. Método modificarPersona

```
@PUT
@Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
@Consumes({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
@Path("{id}")
public Response modificarPersona(@PathParam("id") int id, Persona personaModificada) {
    Persona personaEncontrada = personaService.encontrarPersonaPorId(new Persona(id));
    if (personaEncontrada != null) {
        personaService.modificarPersona(personaModificada);
        return Response.ok().entity(personaModificada).build();
    } else {
        return Response.status(Response.Status.NOT_FOUND).build();
    }
}
```



Este método modifica una persona existente si se encuentra por ID.

7. Método eliminarPersona

```

@DELETE
@Path("/{id}")
public Response eliminarPersonaPorId(@PathParam("id") int id) {
    personaService.eliminarPersona(new Persona(id));
    return Response.ok().build();
}

```

✗ Elimina una persona por su ID. Por ejemplo:

DELETE /webservice/personas/1

📁 Archivo PersonaServiceRS.java completo:

```

package sga.servicio;

import java.util.List;
import jakarta.ejb.Stateless;
import jakarta.inject.Inject;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.*;
import jakarta.ws.rs.core.Response.Status;
import sga.domain.Persona;

@Path("/personas")
@Stateless
public class PersonaServiceRS {

    @Inject
    private PersonaService personaService;

    @GET
    @Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
    public List<Persona> listarPersonas() {
        return personaService.listarPersonas();
    }

    @GET
    @Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
    @Path("/{id}")//hace referencia a /personas/{id}
    public Persona encontrarPersonaPorId(@PathParam("id") int id) {
        return personaService.encontrarPersonaPorId(new Persona(id));
    }

    @POST

```



```
@Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
@Consumes({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
public Response agregarPersona(Persona persona) {
    try {
        personaService.registrarPersona(persona);
        return Response.ok().entity(persona).build();
    } catch (Exception e) {
        e.printStackTrace(System.out);
        return Response.status(Status.INTERNAL_SERVER_ERROR).build();
    }
}

@PUT
@Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
@Consumes({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
@Path("/{id}")
public Response modificarPersona(@PathParam("id") int id, Persona personaModificada) {
    try {
        Persona persona = personaService.encontrarPersonaPorId(new Persona(id));
        if (persona != null) {
            personaService.modificarPersona(personaModificada);
            return Response.ok().entity(personaModificada).build();
        } else {
            return Response.status(Status.NOT_FOUND).build();
        }
    } catch (Exception e) {
        e.printStackTrace(System.out);
        return Response.status(Status.INTERNAL_SERVER_ERROR).build();
    }
}

@DELETE
@Path("/{id}")
public Response eliminarPersonaPorId(@PathParam("id") int id) {
    try {
        personaService.eliminarPersona(new Persona(id));
        return Response.ok().build();
    } catch (Exception e) {
        e.printStackTrace(System.out);
        return Response.status(404).build();
    }
}
```



Paso 4: Modificar la clase `Persona.java`

📁 Ruta: `src/main/java/mx/com/gm/sga/domain/Persona.java`

✚ Agrega la anotación:

```
@XmlRootElement
```

📄 Esto permite que la clase pueda transformarse en XML.

📁 Archivo `Persona.java` completo:

```
package sga.domain;

import jakarta.persistence.*;
import jakarta.validation.constraints.Size;
import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlRootElement;
import jakarta.xml.bind.annotation.XmlTransient;
import java.io.Serializable;
import java.util.List;

@Entity
@NamedQueries({
    @NamedQuery(name = "Persona.findAll", query = "SELECT p FROM Persona p"),
    @NamedQuery(name = "Persona.findByIdPersona", query = "SELECT p FROM Persona p WHERE p.idPersona = :idPersona"),
    @NamedQuery(name = "Persona.findByName", query = "SELECT p FROM Persona p WHERE p.nombre = :nombre"),
    @NamedQuery(name = "Persona.findByApellido", query = "SELECT p FROM Persona p WHERE p.apellido = :apellido"),
    @NamedQuery(name = "Persona.findByEmail", query = "SELECT p FROM Persona p WHERE p.email = :email"),
    @NamedQuery(name = "Persona.findByTelefono", query = "SELECT p FROM Persona p WHERE p.telefono = :telefono"))
@XmlAccessorType(XmlAccessType.FIELD)
@XmlRootElement
public class Persona implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "id_persona")
```

```

private Integer idPersona;
@Size(max = 45)
private String nombre;
@Size(max = 45)
private String apellido;
// @Pattern(regexp="[a-z0-9!#$%&'*/=?^_`{|}~-]+(?:\\. [a-z0-9!#$%&'*/=?^_`{|}~-]+)*@(?:[a-z0-9](?:[a-z0-9]*[a-z0-9])?\\.)+[a-z0-9](?:[a-z0-9]*[a-z0-9])?", message="Invalid email")//if the field
contains email address consider using this annotation to enforce field validation
@Size(max = 45)
private String email;
@Size(max = 45)
private String telefono;

@XmlTransient
@OneToMany(mappedBy = "persona", cascade = CascadeType.ALL)
private List<Usuario> usuarioList;

public Persona() {
}

public Persona(String nombre, String apellido, String email, String telefono) {
    this.nombre = nombre;
    this.apellido = apellido;
    this.email = email;
    this.telefono = telefono;
}

public Persona(Integer idPersona) {
    this.idPersona = idPersona;
}

public Integer getIdPersona() {
    return idPersona;
}

public void setIdPersona(Integer idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

```

```
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}

public List<Usuario> getUsuarioList() {
    return usuarioList;
}

public void setUsuarioList(List<Usuario> usuarioList) {
    this.usuarioList = usuarioList;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (idPersona != null ? idPersona.hashCode() : 0);
    return hash;
}

@Override
public boolean equals(Object object) {
    // TODO: Warning - this method won't work in the case the id fields are not set
    if (!(object instanceof Persona)) {
```

```

        return false;
    }
    Persona other = (Persona) object;
    if ((this.idPersona == null && other.idPersona != null) || (this.idPersona != null &&
!this.idPersona.equals(other.idPersona))) {
        return false;
    }
    return true;
}

@Override
public String toString() {
    return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" + apellido +
", email=" + email + ", telefono=" + telefono + '}';
}
}

```



Paso 5: Configurar el archivo `web.xml`



Ruta: `src/main/webapp/WEB-INF/web.xml`



Agrega la siguiente configuración:

```

<servlet>
  <servlet-name>JerseyWebApplication</servlet-name>
  <servlet-class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>
  <init-param>
    <param-name>jersey.config.server.provider.packages</param-name>
    <param-value>sga.servicio</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>JerseyWebApplication</servlet-name>
  <url-pattern>/webservice/*</url-pattern>
</servlet-mapping>

```



Esto indica que cualquier ruta que empiece con `/webservice/` será manejada por Jersey.



Archivo `web.xml` completo:

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee">
```

```

    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee https://jakarta.ee/xml/ns/jakartaee/web-
app_6_0.xsd"
    version="6.0">

    <context-param>
        <param-name>jakarta.faces.PROJECT_STAGE</param-name>
        <param-value>Development</param-value>
    </context-param>

    <welcome-file-list>
        <welcome-file>index.xhtml</welcome-file>
    </welcome-file-list>

    <servlet>
        <servlet-name>JerseyWebApplication</servlet-name>
        <servlet-class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>
        <init-param>
            <param-name>jersey.config.server.provider.packages</param-name>
            <param-value>sga.servicio</param-value>
        </init-param>
        <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>JerseyWebApplication</servlet-name>
        <url-pattern>/webservice/*</url-pattern>
    </servlet-mapping>
</web-app>

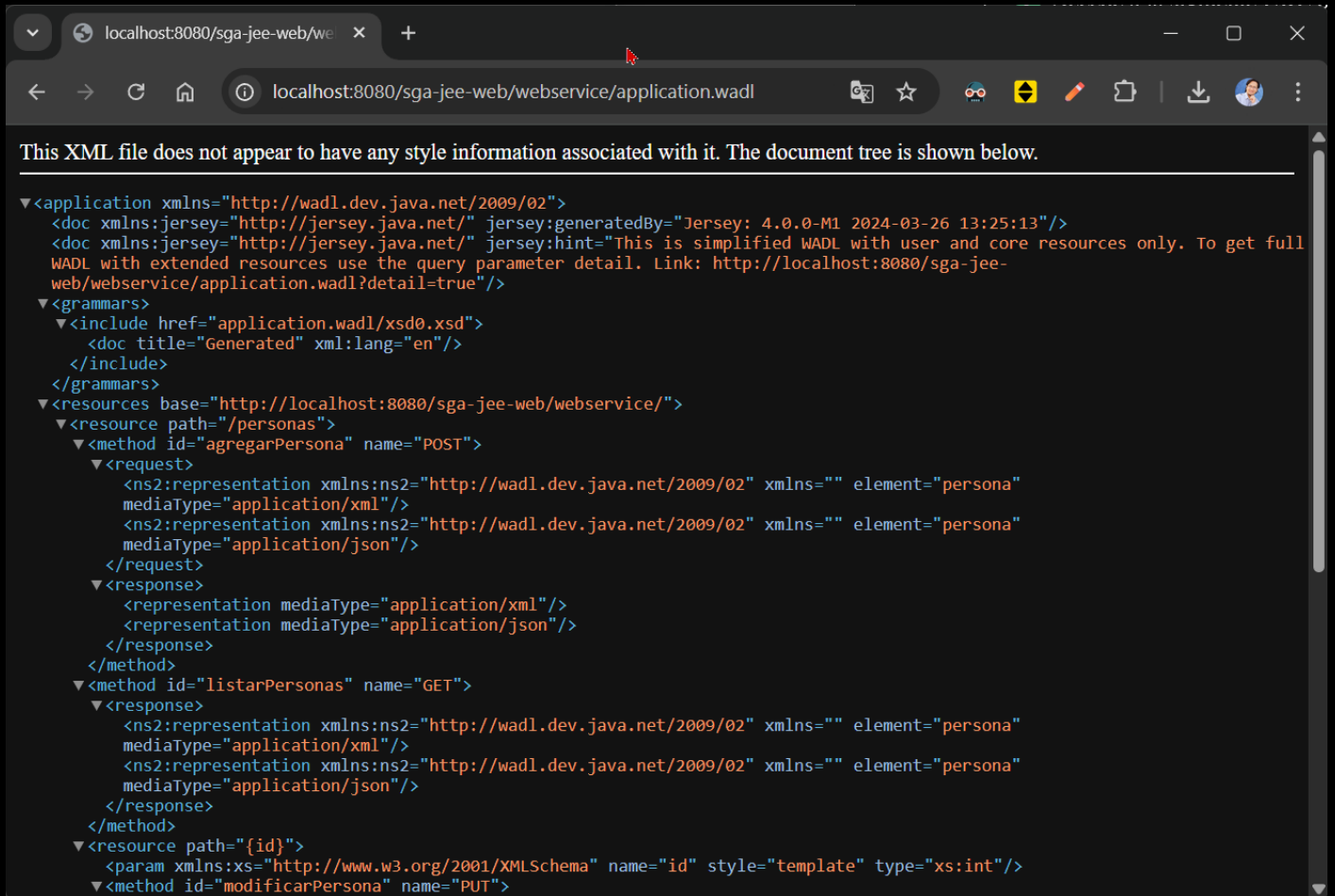
```

✓ Paso 6: Probar el servicio REST

1. ✓ Asegúrate de que GlassFish esté detenido (stop).
2. ✓ Da clic derecho sobre el proyecto y selecciona **Run** (esto levanta Glassfish en automático)
3. ✓ Accede a la siguiente URL para probar:

<http://localhost:8080/sga-jee-web/webservice/application.wadl>

📄 Verás una lista de los endpoints disponibles.



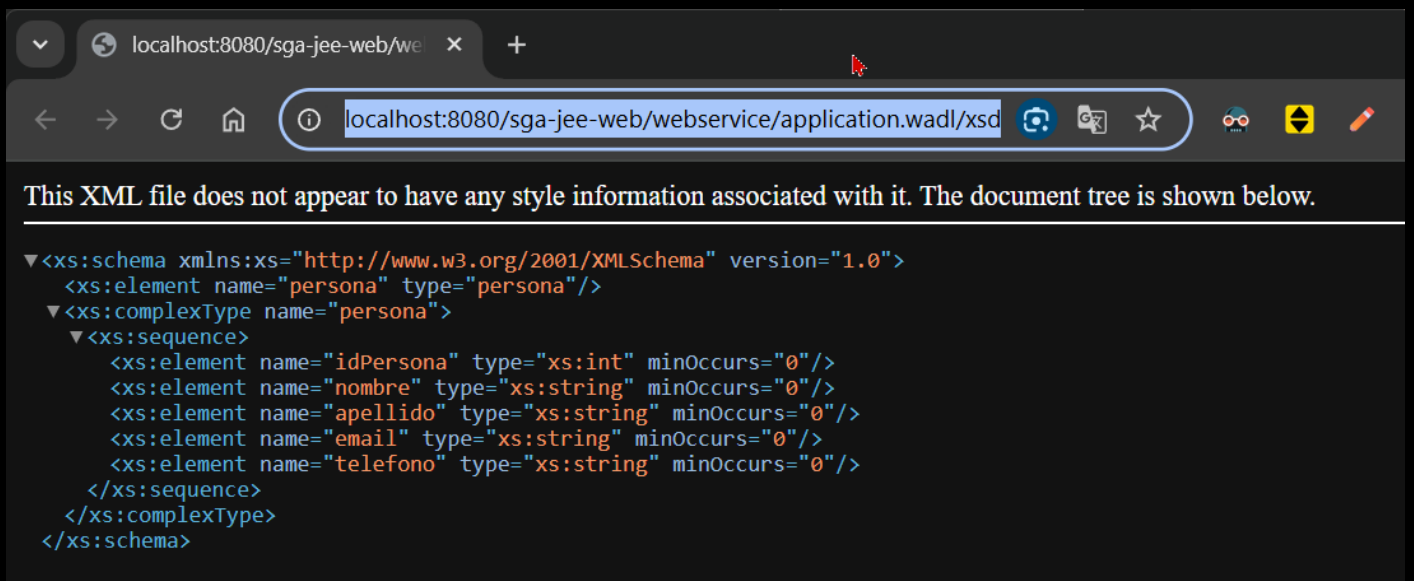
This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<application xmlns="http://wadl.dev.java.net/2009/02">
  <doc xmlns:jersey="http://jersey.java.net/" jersey:generatedBy="Jersey: 4.0.0-M1 2024-03-26 13:25:13"/>
  <doc xmlns:jersey="http://jersey.java.net/" jersey:hint="This is simplified WADL with user and core resources only. To get full WADL with extended resources use the query parameter detail. Link: http://localhost:8080/sga-jee-web/webservice/application.wadl?detail=true"/>
  <grammars>
    <include href="application.wadl/xsd0.xsd">
      <doc title="Generated" xml:lang="en"/>
    </include>
  </grammars>
  <resources base="http://localhost:8080/sga-jee-web/webservice/">
    <resource path="/personas">
      <method id="agregarPersona" name="POST">
        <request>
          <ns2:representation xmlns:ns2="http://wadl.dev.java.net/2009/02" xmlns="" element="persona" mediaType="application/xml"/>
          <ns2:representation xmlns:ns2="http://wadl.dev.java.net/2009/02" xmlns="" element="persona" mediaType="application/json"/>
        </request>
        <response>
          <representation mediaType="application/xml"/>
          <representation mediaType="application/json"/>
        </response>
      </method>
      <method id="listarPersonas" name="GET">
        <response>
          <ns2:representation xmlns:ns2="http://wadl.dev.java.net/2009/02" xmlns="" element="persona" mediaType="application/xml"/>
          <ns2:representation xmlns:ns2="http://wadl.dev.java.net/2009/02" xmlns="" element="persona" mediaType="application/json"/>
        </response>
      </method>
      <resource path="{id}">
        <param xmlns:xs="http://www.w3.org/2001/XMLSchema" name="id" style="template" type="xs:int"/>
        <method id="modificarPersona" name="PUT">

```

También puedes acceder a los xsd de cada tipo, ej. el xsd de tipo persona:

<http://localhost:8080/sga-jee-web/webservice/application.wadl/xsd0.xsd>



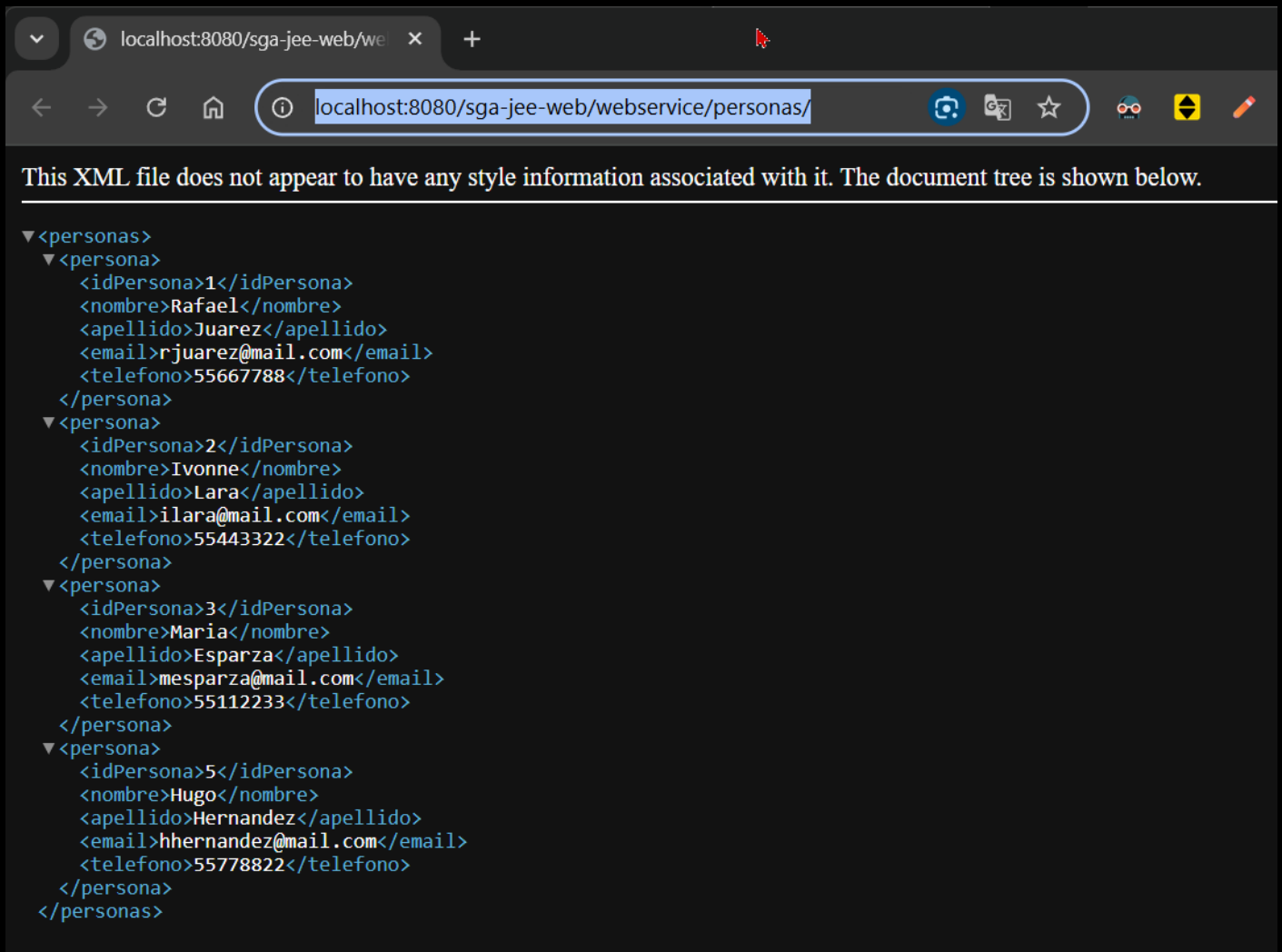
This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0">
  <xs:element name="persona" type="persona"/>
  <xs:complexType name="persona">
    <xs:sequence>
      <xs:element name="idPersona" type="xs:int" minOccurs="0"/>
      <xs:element name="nombre" type="xs:string" minOccurs="0"/>
      <xs:element name="apellido" type="xs:string" minOccurs="0"/>
      <xs:element name="email" type="xs:string" minOccurs="0"/>
      <xs:element name="telefono" type="xs:string" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

Ejemplo de prueba:

Obtener el listado de personas:

<http://localhost:8080/sga-jee-web/webservice/personas/>



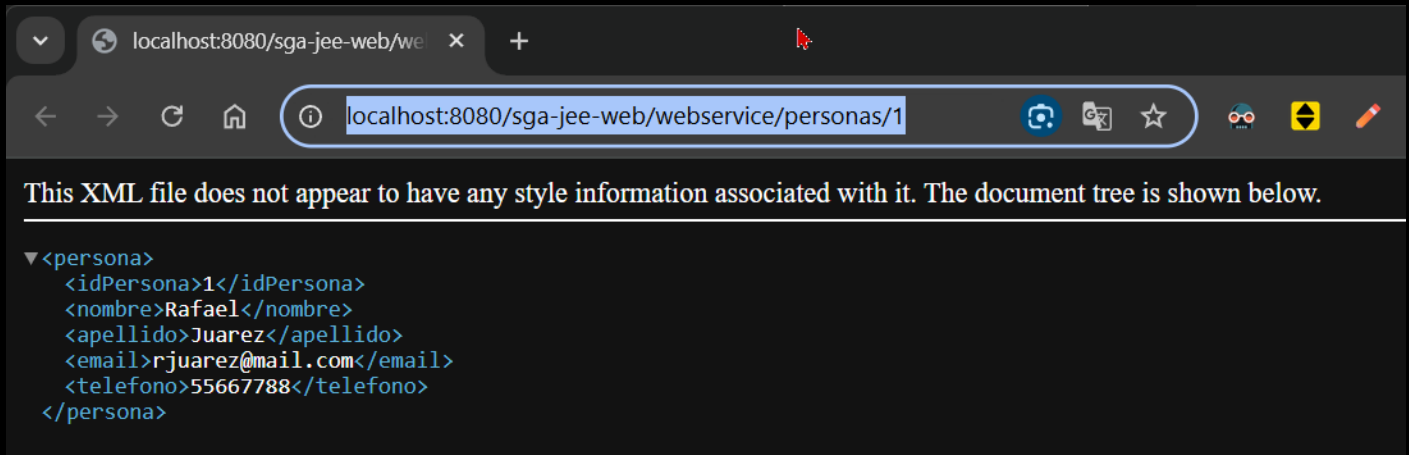
This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<personas>
  <persona>
    <idPersona>1</idPersona>
    <nombre>Rafael</nombre>
    <apellido>Juarez</apellido>
    <email>rjuarez@mail.com</email>
    <telefono>55667788</telefono>
  </persona>
  <persona>
    <idPersona>2</idPersona>
    <nombre>Ivonne</nombre>
    <apellido>Lara</apellido>
    <email>ilara@mail.com</email>
    <telefono>55443322</telefono>
  </persona>
  <persona>
    <idPersona>3</idPersona>
    <nombre>Maria</nombre>
    <apellido>Esparza</apellido>
    <email>mesparza@mail.com</email>
    <telefono>55112233</telefono>
  </persona>
  <persona>
    <idPersona>5</idPersona>
    <nombre>Hugo</nombre>
    <apellido>Hernandez</apellido>
    <email>hhernandez@mail.com</email>
    <telefono>55778822</telefono>
  </persona>
</personas>
```

Obtener persona con ID 1:

<http://localhost:8080/sga-jee-web/webservice/personas/1>

 Devuelve:



Conclusión

Con este ejercicio aprendiste a:

- ✓ Exponer métodos de negocio como servicios REST.
- ✓ Trabajar con anotaciones como @GET, @POST, @PUT, @DELETE.
- ✓ Transformar objetos en XML o JSON usando @XmlRootElement.
- ✓ Configurar Jersey en un proyecto Maven.
- ✓ Consumir endpoints desde el navegador u otras aplicaciones cliente.

 ¡Felicidades por completar este módulo de RESTful Web Services!

Sigue adelante con tu aprendizaje  , ¡el esfuerzo vale la pena!

¡Saludos! 

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)